

NATURAL LANGUAGE PROCESSING

(NLP)

juillet 2020

RNN

Réseaux de neurones récurrents

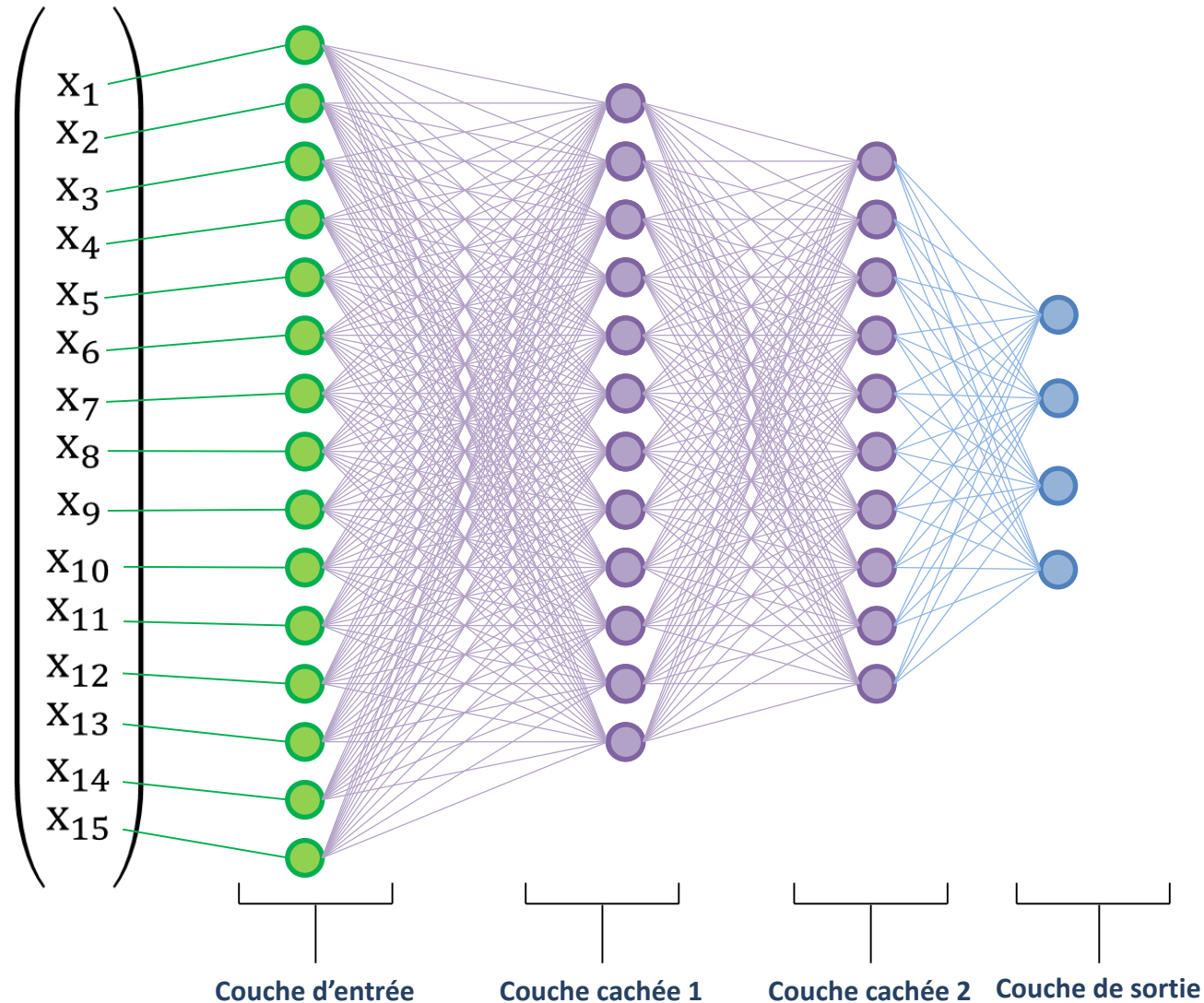
JAOUAD DABOUNOU

FST de Settat

Université Hassan 1^{er}

Réseau de neurones traditionnel

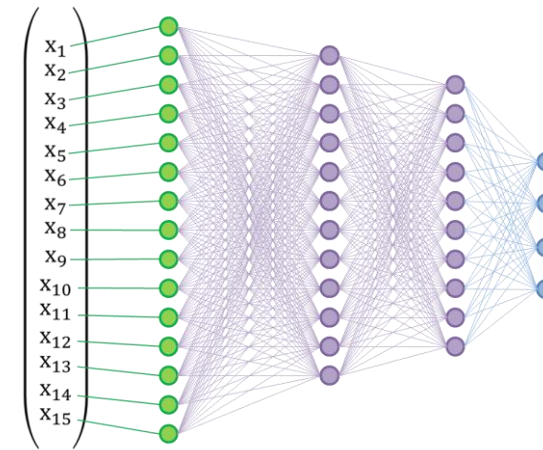
Exemple d'architecture :



Réseau de neurones traditionnel

Limites des architectures traditionnelles :

- Taille fixe de la couche d'entrée
- Taille fixe de la couche de sortie
- Architecture fixe du réseau
- Ne tiennent pas compte de l'ordre des données
- Non adaptées aux séries temporelles



Introduction au RNN

Un réseau de neurones récurrent (RNN, recurrent neural network) est un type de réseau de neurones artificiels principalement utilisé dans la reconnaissance automatique de la parole, dans l'écriture manuscrite et dans le traitement automatique du langage naturel, en particulier dans la traduction automatique.

Les RNN sont conçus de manière à reconnaître les caractéristiques séquentielles et pour prédire le scénario suivant le plus probable.

Ils se distinguent des réseaux de neurones traditionnels en ce sens qu'ils utilisent des boucles de rétroaction pour traiter une séquence de données qui façonne le résultat final, lui-même pouvant être une séquence de données. Ces boucles de rétroaction permettent aux informations de persister, effet souvent assimilé à la mémoire.

Les RNN sont ainsi en mesure de traiter des données séquentielles et temporelles.

 **Concept de mémoire**

RNN et modèles de langue

On rappelle qu'un modèle de langue est :

- une représentation de la manière dont se comprend une langue, se construisent ses phrases et se coordonnent ses mots.
- Un ensemble de régularités linguistiques et de structures latentes utilisées pour s'exprimer dans une langue, porter et communiquer un sens.
- la manifestation de l'ensemble de normes, cultures et habitudes langagières à travers l'ordonnancement des mots et phrases, partagées par une communauté qui s'exprime dans une langue donnée.

Les RNN sont utilisés dans des modèles de langue performants. Ils considèrent à la fois l'entrée textuelle actuelle et une «**unité de contexte**» construite sur ce qu'ils ont vu précédemment pour prédire l'unité suivante.

Modèles de Langue traditionnels

Dans un modèle de langue, le sens d'un mot est lié au contexte de son utilisation. Un modèle de langue calcule alors une probabilité pour une séquence de mots $P(w_1, \dots, w_k)$. On a par exemple:

$P(\text{Omar enfant est un petit}) < P(\text{Omar est un petit enfant})$

Les modèles vont se différencier par la manière dont ils calculent cette probabilité et par la longueur des séquences considérées. Ainsi, traditionnellement, pour les unigrammes et bigrammes

$$p(w_2|w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}, \quad p(w_3|w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$$

Pour les n-grammes

$$p(w_n|w_1^{n-1}) = \frac{\text{count}(w_1^n)}{\text{count}(w_1^{n-1})}$$

Enfin :

$$p(w_1^n) = p(w_1) p(w_2|w_1) p(w_3|w_1^2) \dots p(w_n|w_1^{n-1})$$

Les RNN améliorent cette prise en compte du contexte et acceptent des tailles de fenêtres variables pour le contexte.

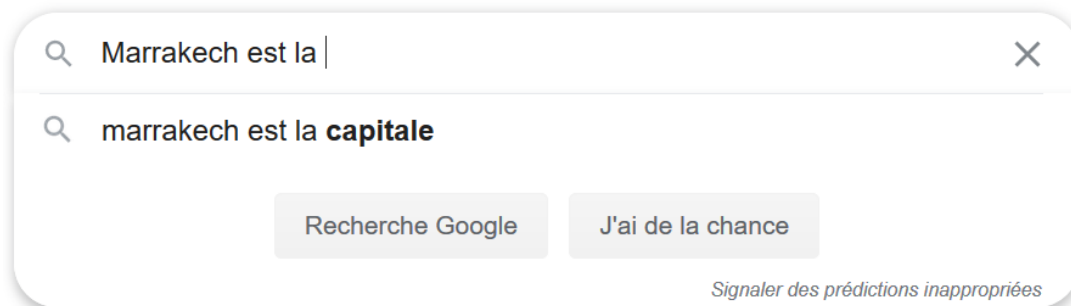
Prédire le mot suivant dans une séquence

Les RNN (et des algorithmes à base de RNN) ont montré leur utilité dans de nombreuses applications. Ils prédisent avec efficacité le mot suivant dans une phrase:

Marrakech est la capitale touristique du Maroc

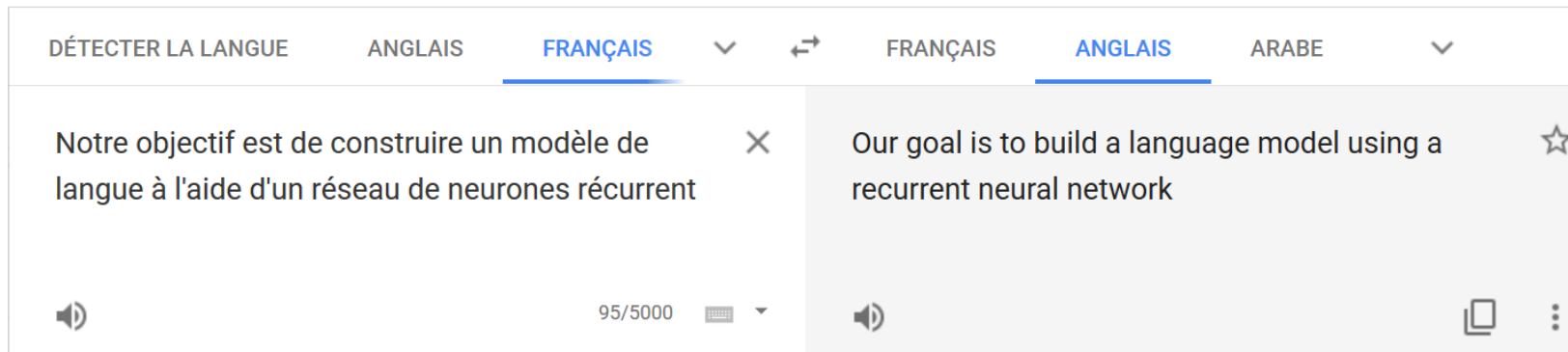
Etant donnés ces mots

Prédire le mot suivant



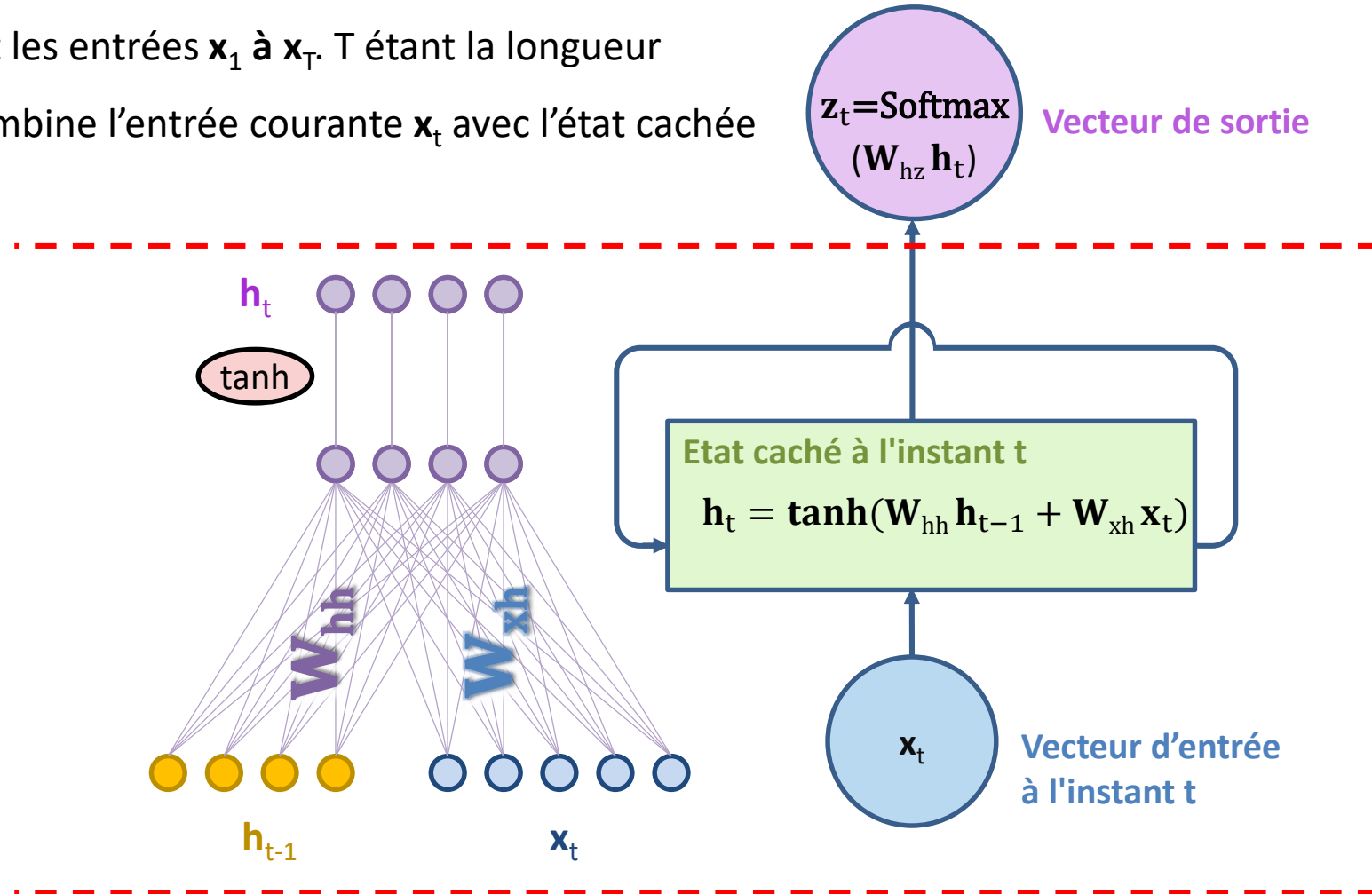
Traduction automatique

La traduction automatique est basée principalement sur les réseaux de neurones récurrents.



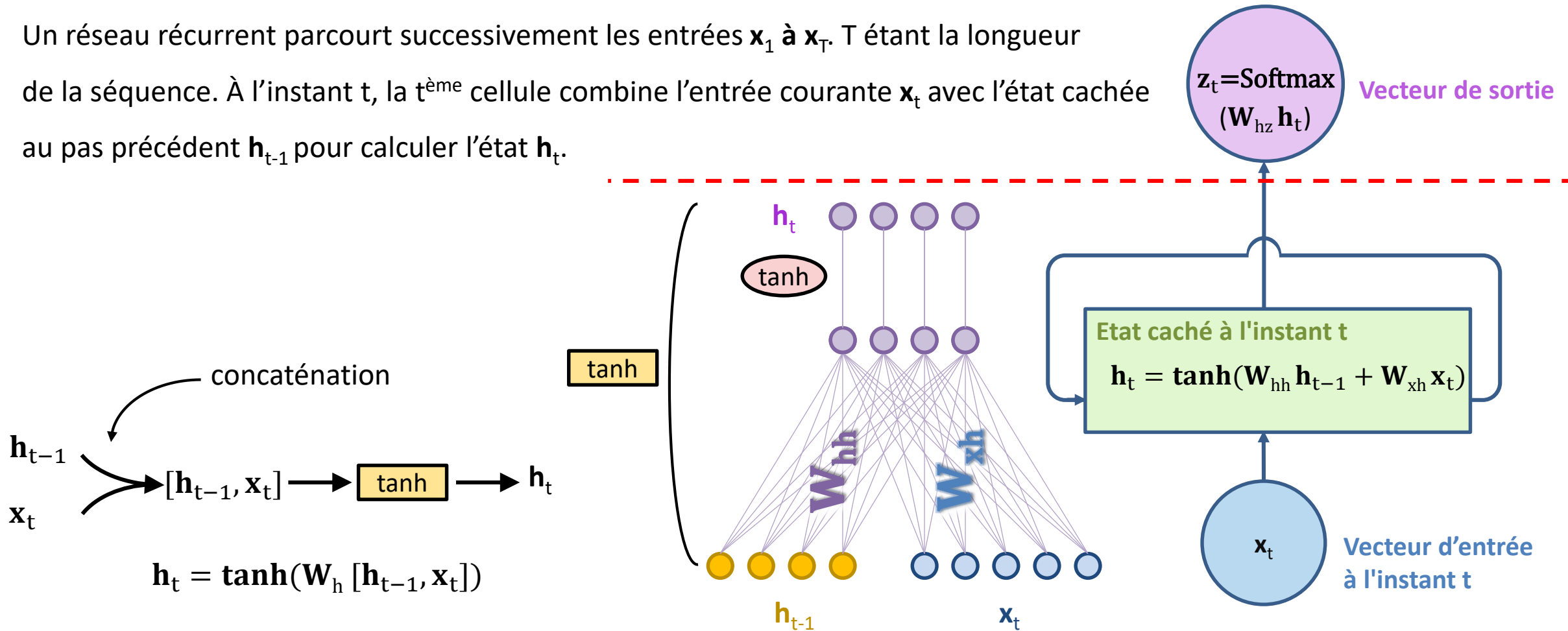
Réseaux Récurrents

Un réseau récurrent parcourt successivement les entrées $\mathbf{x}_1$ à $\mathbf{x}_T$. T étant la longueur de la séquence. À l'instant t , la $t^{\text{ème}}$ cellule combine l'entrée courante $\mathbf{x}_t$ avec l'état caché au pas précédent $\mathbf{h}_{t-1}$ pour calculer l'état $\mathbf{h}_t$.



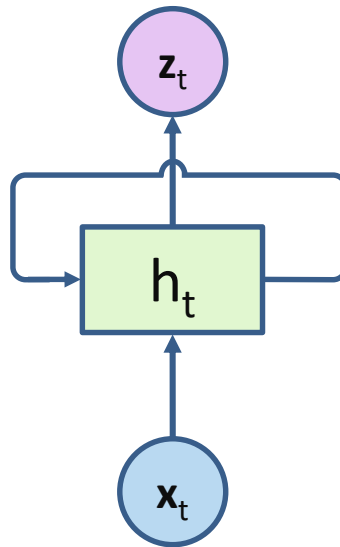
Réseaux Récurrents

Un réseau récurrent parcourt successivement les entrées $\mathbf{x}_1$ à $\mathbf{x}_T$. T étant la longueur de la séquence. À l'instant t , la $t^{\text{ème}}$ cellule combine l'entrée courante $\mathbf{x}_t$ avec l'état caché au pas précédent $\mathbf{h}_{t-1}$ pour calculer l'état $\mathbf{h}_t$.



Réseaux Récurrents

Un réseaux récurrent parcourt successivement les entrées x_1 à x_T . À l'instant t , la $t^{\text{ème}}$ cellule combine l'entrée courante x_t avec l'état cachée au pas précédent h_{t-1} pour calculer l'état h_t .



La ligne qui entoure la couche cachée du réseau de neurones récurrent s'appelle la boucle temporelle.

Elle indique que la couche cachée génère non seulement une sortie, mais que la sortie est renvoyée en tant qu'entrée dans la même couche à l'instant suivant.

Réseaux Récurrents

L'expression :

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$

est parfois remplacée par :

$$\mathbf{h}_t = \mathbf{f}_W(\mathbf{h}_{t-1}, \mathbf{x}_t) = \tanh(\mathbf{W}_h \text{concat}(\mathbf{h}_{t-1}, \mathbf{x}_t))$$

Ou encore

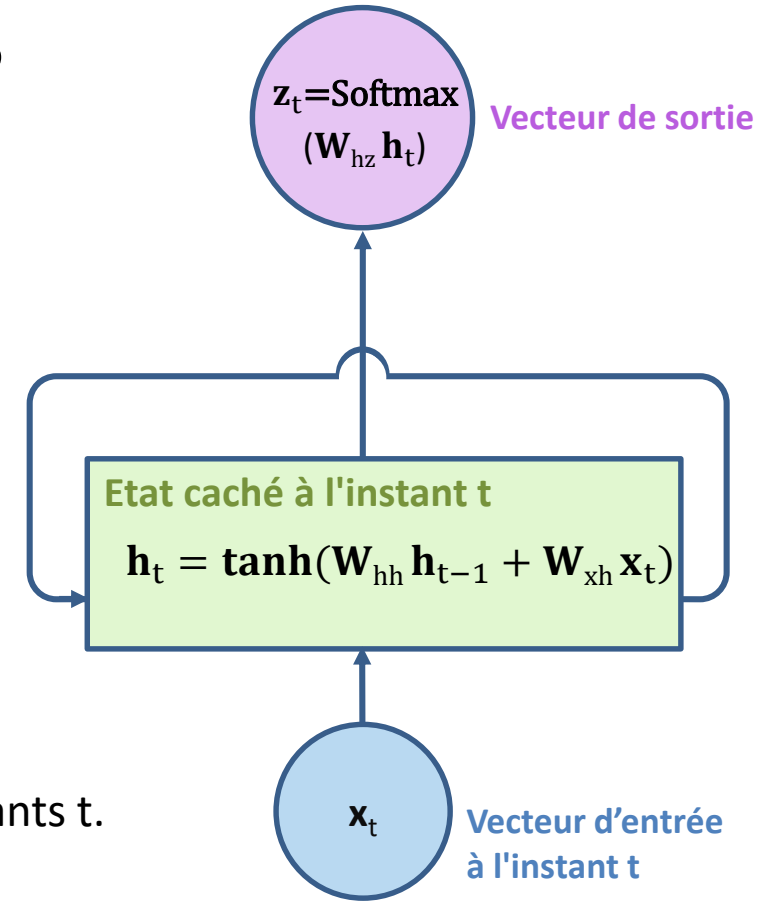
$$\mathbf{h}_t = \mathbf{f}_W(\mathbf{h}_{t-1}, \mathbf{x}_t) = \tanh(\mathbf{W}_h [\mathbf{h}_{t-1}, \mathbf{x}_t])$$

A noter que les paramètres $\mathbf{W}_{hh}$ et $\mathbf{W}_{xh}$ (ou $\mathbf{W}_h$) sont les mêmes à tous les instants t .

Pour $t=0$, on utilise un état caché initialisé aléatoirement.

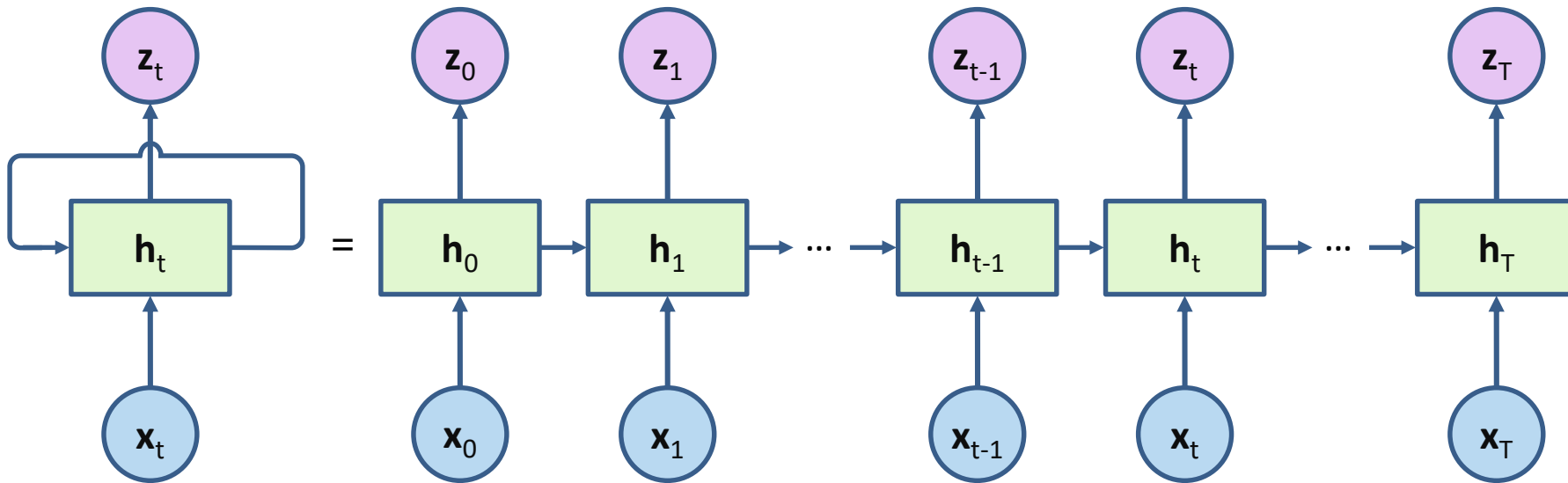
$\mathbf{z}_t$ est une distribution de probabilité sur le vocabulaire elle prédit un mot en sortie.

Parfois on omet la fonction softmax. La sortie devient simplement: $\mathbf{z}_t = \mathbf{W}_{hz}^t \mathbf{h}_t$



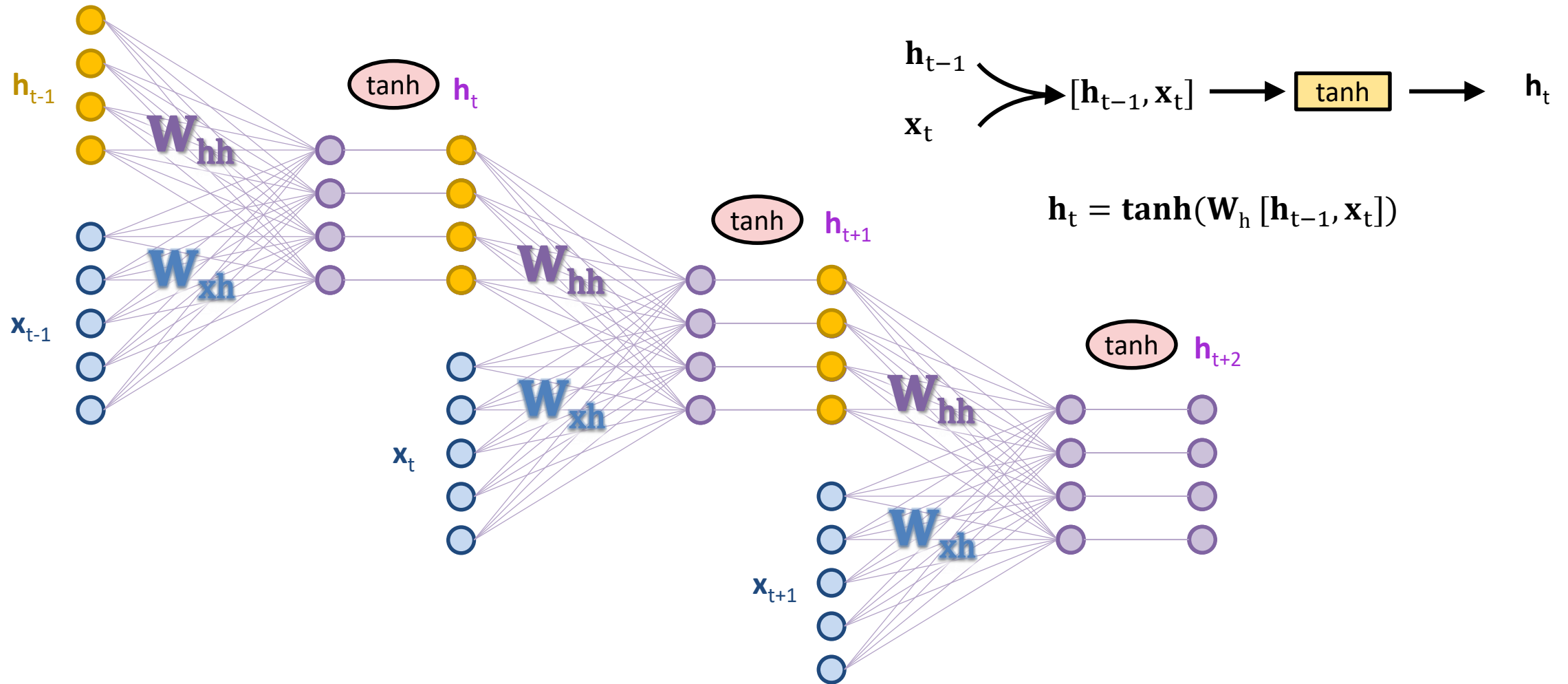
Dérouler un réseau récurrent

On propose souvent une présentation du réseau récurrent déroulé pour mieux comprendre son fonctionnement.



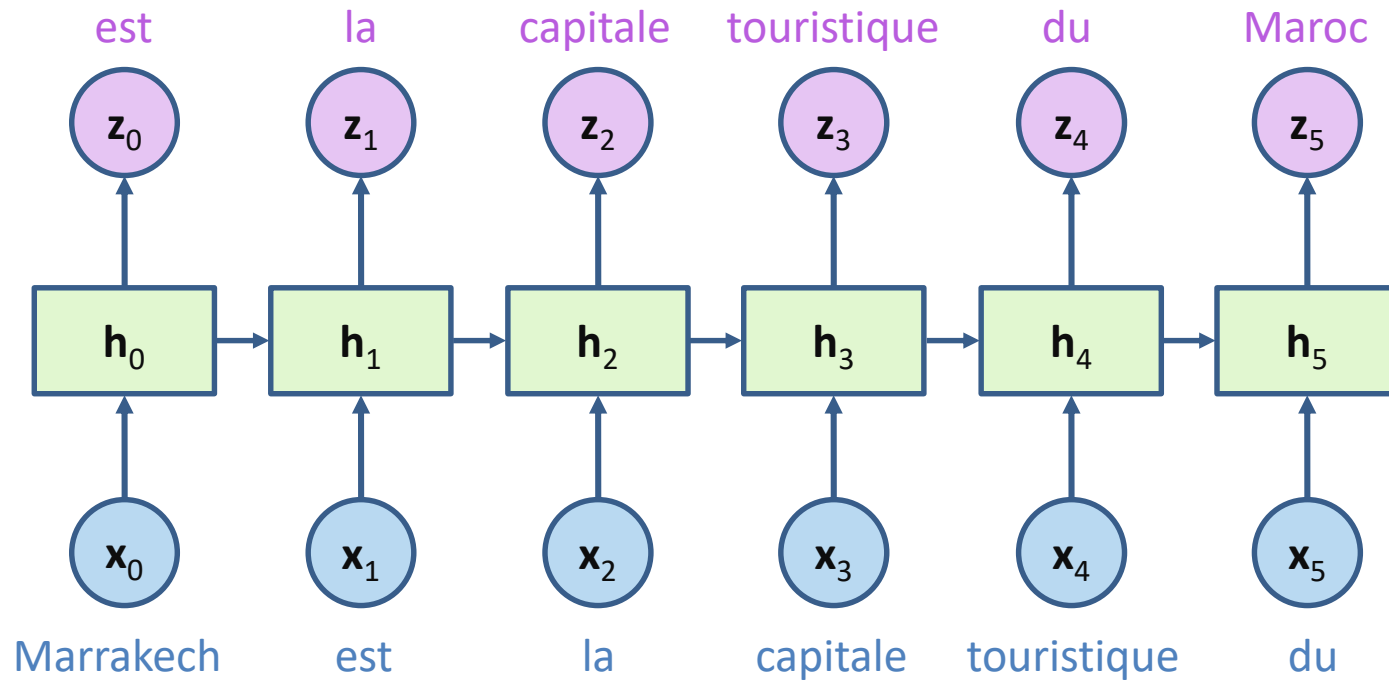
Réseau RNN traditionnel

Un RNN peut ainsi être assimilé à un feedforward neural network comme le montre la figure suivante :



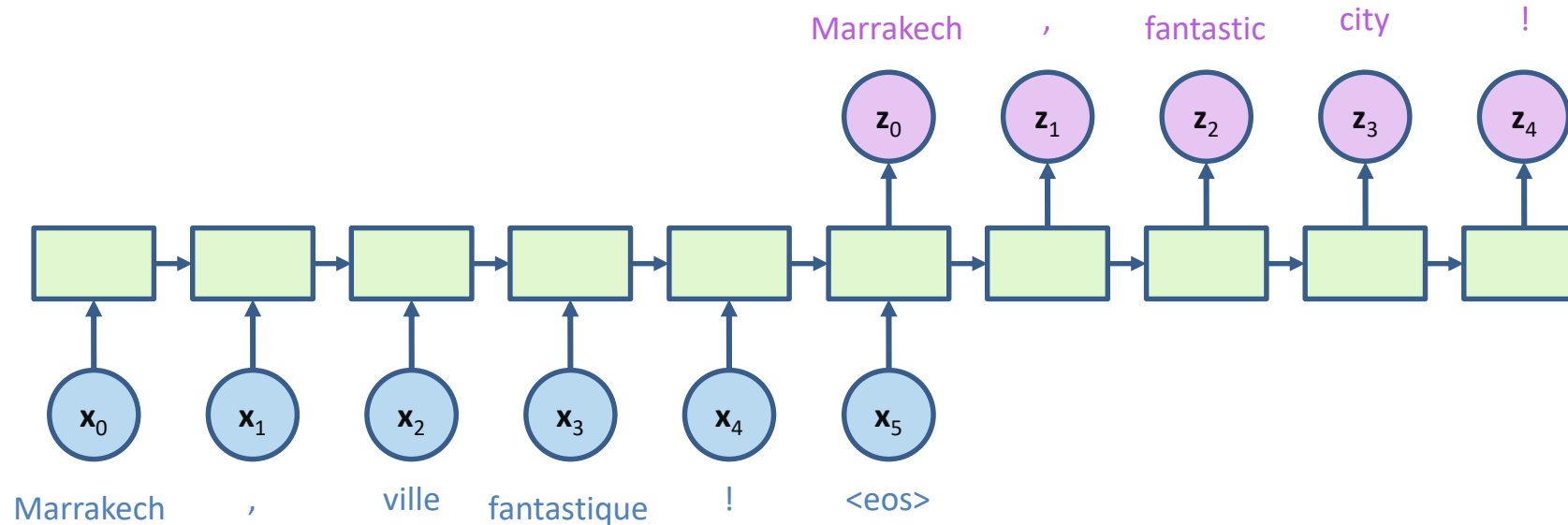
Prédire le mot suivant

Un réseau récurrent peut par exemple générer une séquence de mots à partir d'un premier mot entré, « Marrakech » dans l'exemple suivant :



Réseau récurrent pour la traduction

Un réseau récurrent peut aussi être utilisé dans la traduction automatique. Nous présentons ci-dessous une architecture simplifiée basée sur un réseau récurrent. Nous allons revenir à cet exemple avec plus de détails (encodeur/décodeur, modèle de traduction,...) dans de futures présentations.

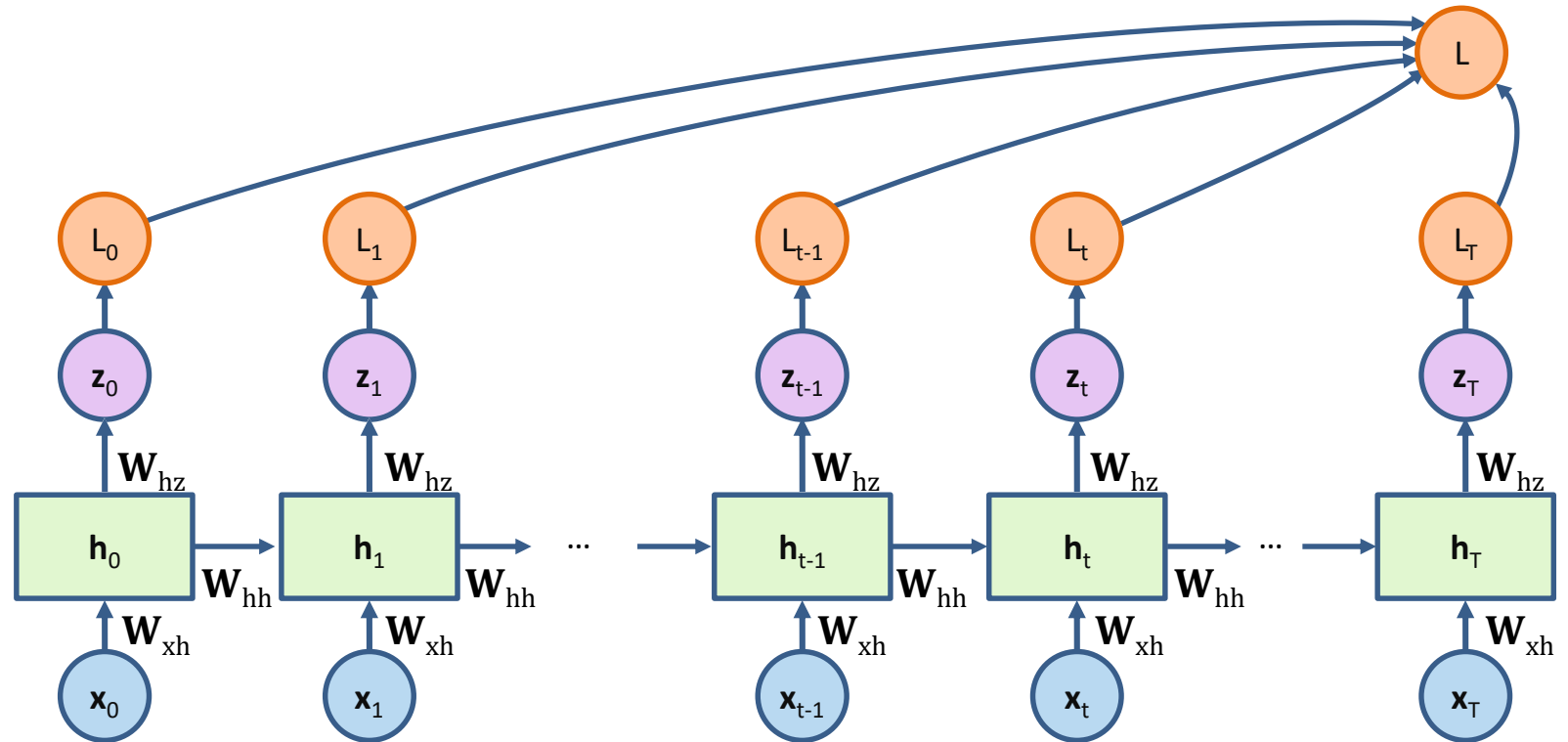


Les entrées du réseau sont une représentation vectorielle des mots (One-hot encoding, word embedding,...) et non les mots eux-mêmes.

Fonction perte du réseau récurrent

On se place dans un problème de classification de séquences. A chaque pas de temps t , le réseau reçoit une entrée $\mathbf{x}_t$ et retourne une sortie $\mathbf{z}_t$. Soit $\mathbf{y}_t$ la sortie souhaitée. Soit θ le vecteur des paramètres du réseau obtenus à partir de $\mathbf{W}_{hh}$, $\mathbf{W}_{xh}$ et $\mathbf{W}_{hz}$. Le modèle doit minimiser la log-vraisemblance négative :

$$LL(\theta) = - \sum_{t=1}^T \log P(\mathbf{y}_t | \mathbf{x}_t)$$

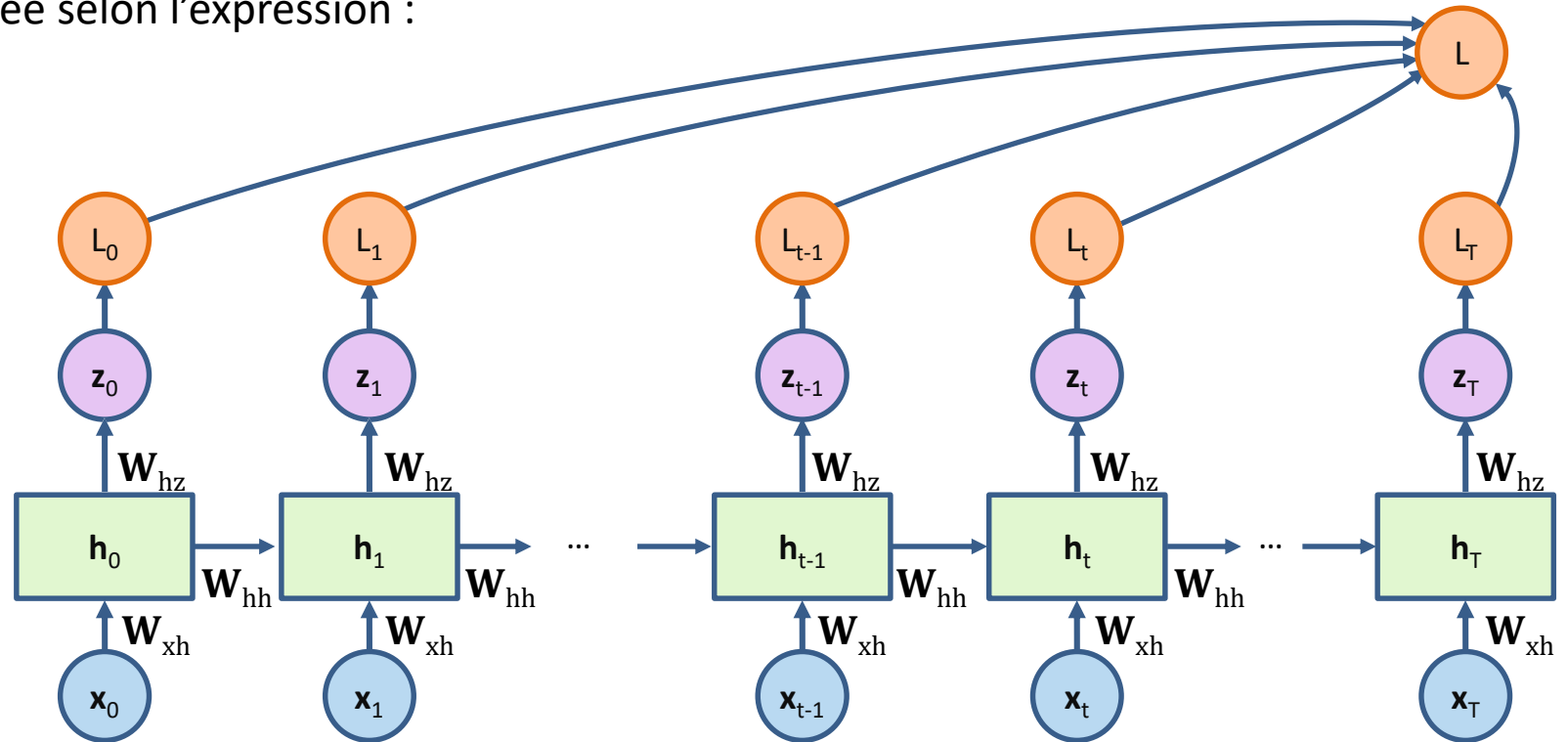


Fonction perte du réseau récurrent

Pour illustrer les concepts introduits, on se place dans un problème de classification de séquences de mots. Pour chaque pas de temps $t=1,T$, soient $\mathbf{x}_t$ l'entrée du réseau, $\mathbf{z}_t$ le mot retourné et $\mathbf{y}_t$ le mot attendu.

On a $\mathbf{z}_t \in \mathbf{R}^V$ où V est la taille du vocabulaire. A chaque pas de temps $t=1,T$ (donc pour chaque mot) on calcule la fonction de perte d'entropie croisée selon l'expression :

$$L_t(\theta) = - \sum_{k=1}^V y_{t,k} \log(z_{t,k})$$

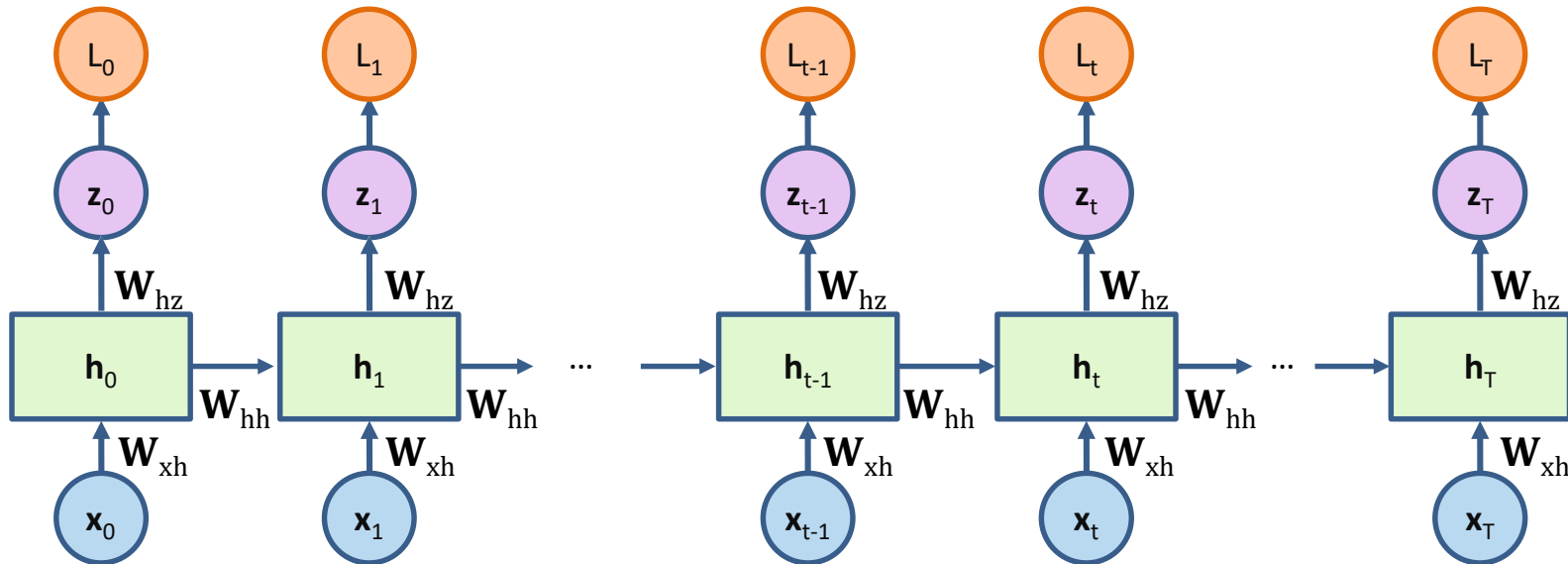


Apprentissage du réseau récurrent

L'apprentissage se fait par rétropropagation du gradient de la fonction de perte sur une séquence ou sur un ensemble de séquences (si taille du batch > 1) :

$$\frac{\partial L(\theta)}{\partial \theta} = \sum_{t=1}^T \frac{\partial L_t(\theta)}{\partial \theta}$$

$$\frac{\partial L_t(\theta)}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial \mathbf{z}_t} \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \theta}$$



$$L_t(\theta) = - \sum_{k=1}^v \mathbf{y}_{t,k} \log(\mathbf{z}_{t,k})$$

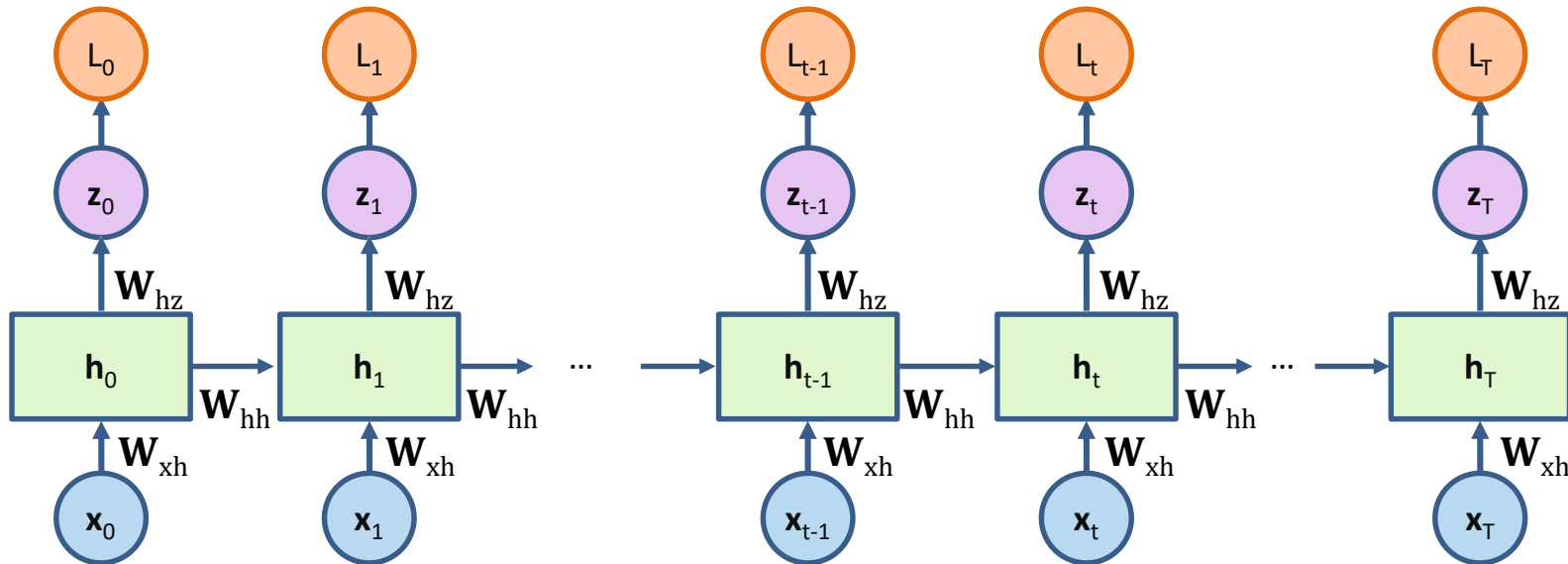
$$\mathbf{z}_t = \text{Softmax}(\mathbf{W}_{hz} \mathbf{h}_t)$$

Apprentissage du réseau récurrent

L'apprentissage se fait par rétropropagation du gradient de la fonction de perte sur une séquence ou sur un ensemble de séquences (si taille du batch > 1) :

$$\frac{\partial L(\theta)}{\partial \theta} = \sum_{t=1}^T \frac{\partial L_t(\theta)}{\partial \theta}$$

$$\frac{\partial L_t(\theta)}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial h_t} \frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial \mathbf{z}_t} \frac{\partial \mathbf{z}_t}{\partial h_t} \frac{\partial h_t}{\partial \theta}$$



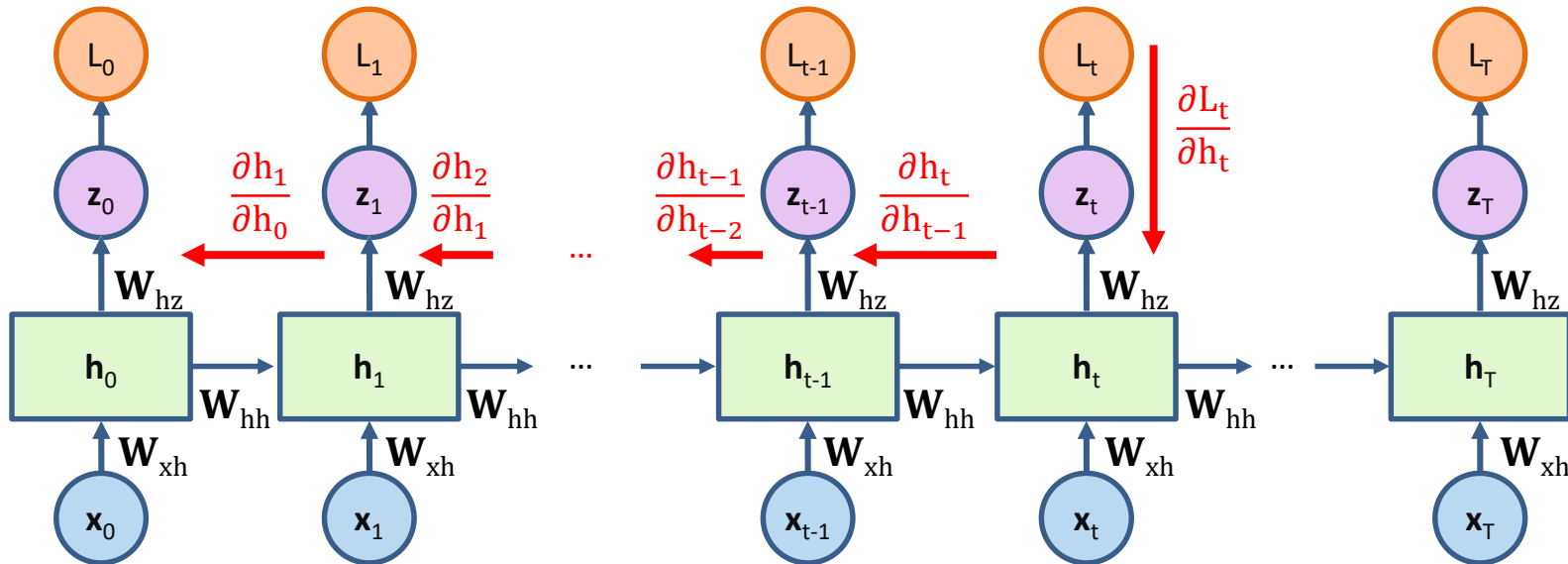
$$\frac{\partial h_t}{\partial \theta} = \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \dots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial \theta}$$

Apprentissage du réseau récurrent

L'apprentissage se fait par rétropropagation du gradient de la fonction de perte sur une séquence ou sur un ensemble de séquences (si taille du batch > 1) :

$$\frac{\partial L(\theta)}{\partial \theta} = \sum_{t=1}^T \frac{\partial L_t(\theta)}{\partial \theta}$$

$$\frac{\partial L_t(\theta)}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial h_t} \frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial z_t} \frac{\partial z_t}{\partial h_t} \frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial z_t} \frac{\partial z_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \dots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial \theta}$$



On parle alors de rétropropagation à travers le temps (Back prop through time BPTT).

Vanishing Gradients du RNN

On a :

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$

On pose $\phi_t = \mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t$, ainsi $\mathbf{h}_t = \tanh(\phi_t)$

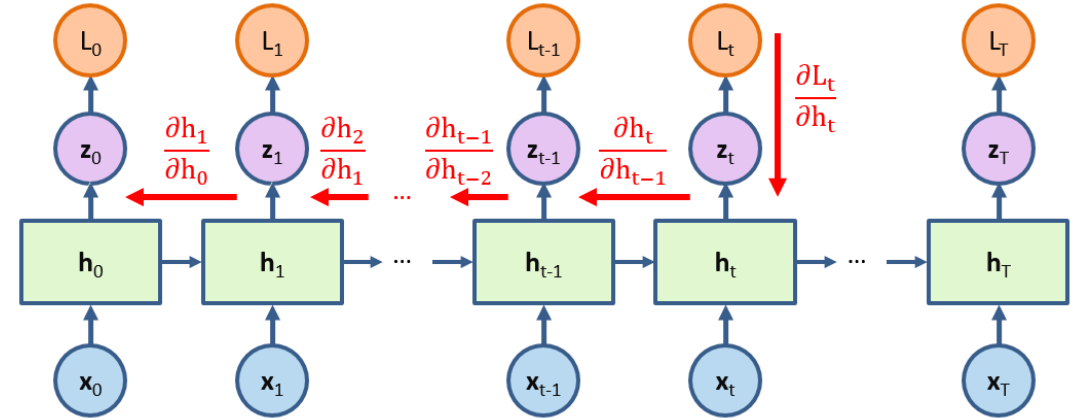
$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \frac{\partial \mathbf{h}_t}{\partial \phi_t} \frac{\partial \phi_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(\tanh'(\phi_t)) \mathbf{W}_{hh}$$

$$\tanh'(x) = 1 - \tanh^2(x)$$

est bornée par $\alpha=1$. Si on avait choisi une sigmoïde σ à la place de $\tanh$, alors σ' serait bornée par $\alpha=0.25$ par exemple.

On peut ainsi écrire :

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \right\| \leq (\alpha \|\mathbf{W}_{hh}\|)^t$$

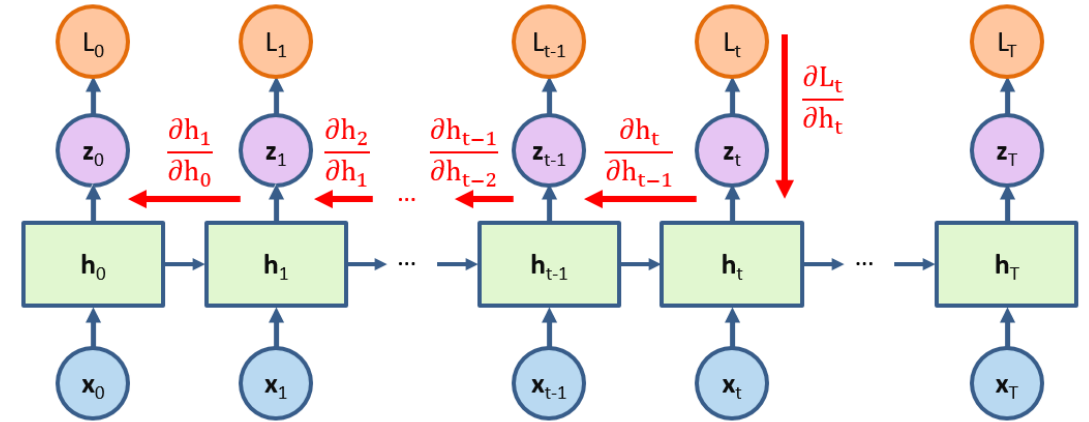


Vanishing Gradients du RNN

On a :

$$\frac{\partial L_t(\theta)}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial h_t} \frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial \mathbf{z}_t} \frac{\partial \mathbf{z}_t}{\partial h_t} \frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial \mathbf{z}_t} \frac{\partial \mathbf{z}_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial \theta}$$

$$\left\| \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \right\| \leq (\alpha \|\mathbf{W}_{hh}\|)^t$$



Deux situations se présentent :

1. $\alpha \|\mathbf{W}_{hh}\| < 1$, alors il y a disparition du gradient (vanishing gradient):

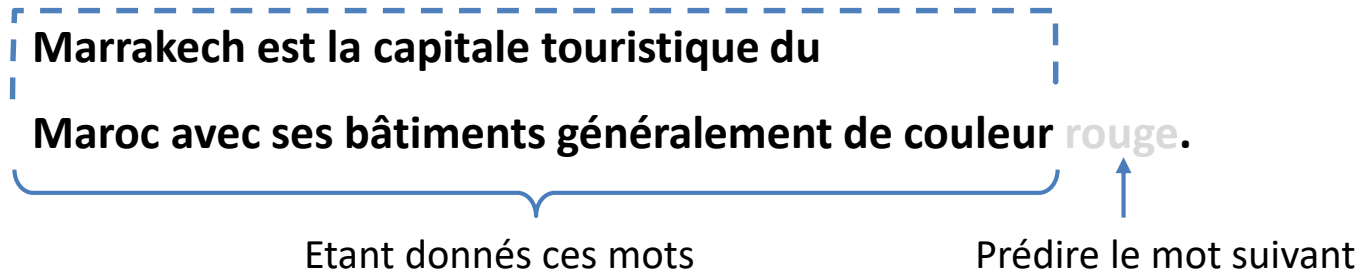
Donc $\frac{\partial L_t(\theta)}{\partial \theta}$ devient presque nul surtout lorsque T est grand (séquence longue)

2. $\alpha \|\mathbf{W}_{hh}\| > 1$, alors il y a explosion du gradient (exploding gradient).

Dans les deux cas, le réseau récurrent trouve des difficultés à apprendre.

Limites des réseaux RNN

Les RNN ne permettent pas de mémoriser les dépendances à long ou très long terme. Par exemple, dans la phrase suivante, le réseau récurrent risque de ne pas considérer le lien entre les mots « Marrakech » et « rouge » :



Solutions au Pb du vanishing gradient

Pour éviter la disparition ou l'explosion du gradient :

- Arrêter la rétropropagation après K termes, même si cela ne permet pas de mettre à jour tous les poids,
- Maintenir le gradient dans un intervalle fixé à l'avance,
- Pénaliser ou appliquer des techniques pour ajuster le gradient

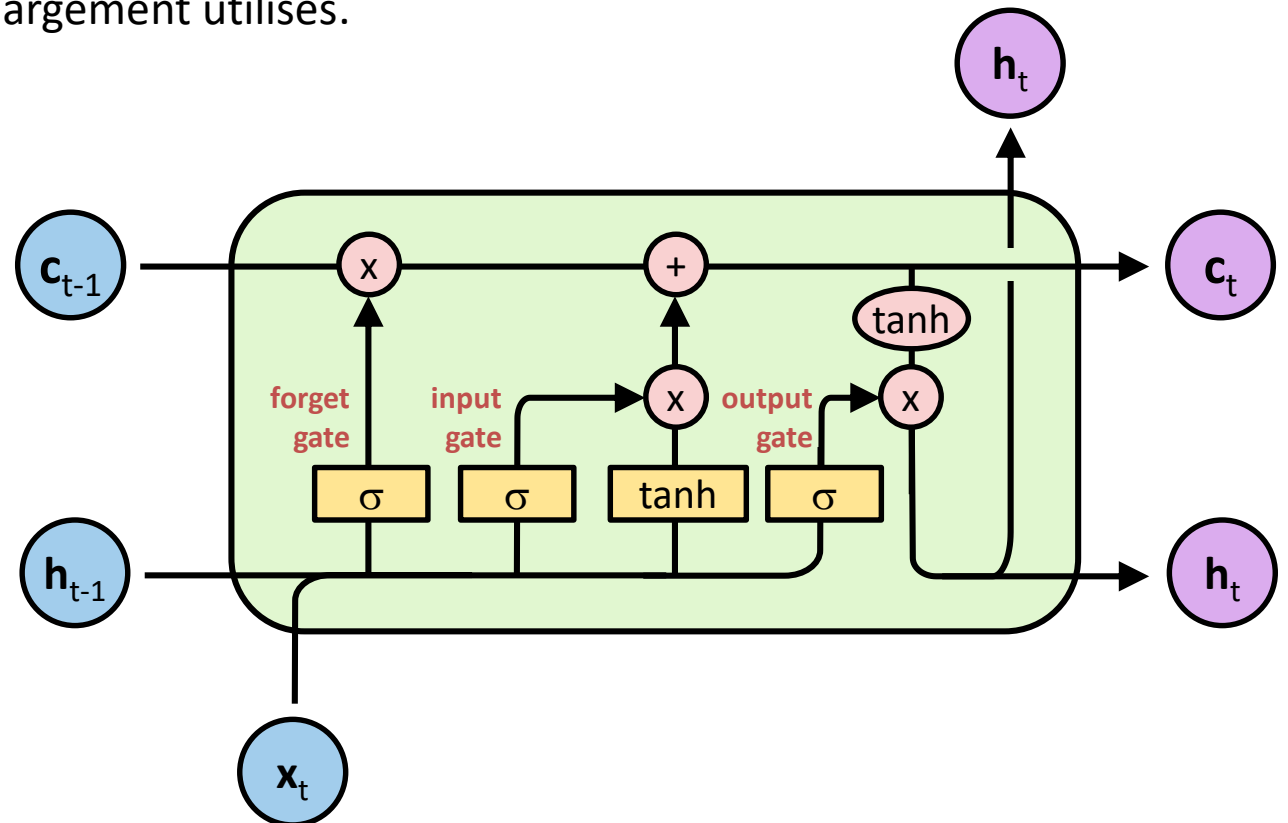
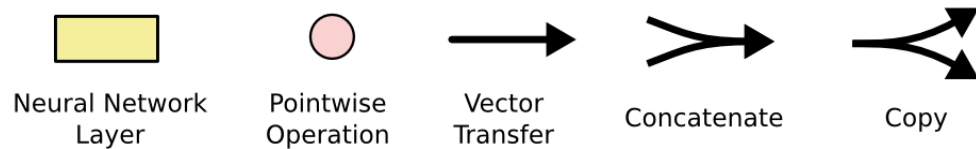
Reste que les LSTM sont considérés comme incontournables pour la mise en œuvre des RNN. Ces réseaux seront décrits dans une future présentation.

Réseaux LSTM

Les réseaux LSTM (Long Short Term Memory ou mémoire à long terme et à court terme) sont un type spécial de RNN, capable d'apprendre les dépendances à long terme. Ils ont été introduits par Hochreiter et Schmidhuber en 1997, et ont été par la suite affinés et popularisés à travers plusieurs travaux. Ils fonctionnent extrêmement bien sur une grande variété de problèmes et sont maintenant largement utilisés.

Nous allons présenter les LSTM dans une suite de transformations des réseaux RNN standard.

Nous utiliserons alors les symboles suivants^(*) :

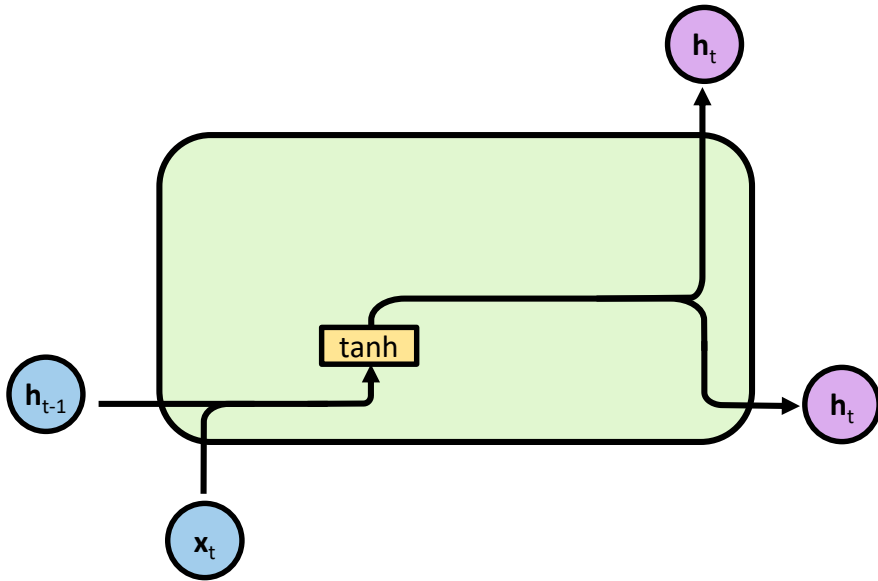


^(*)<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

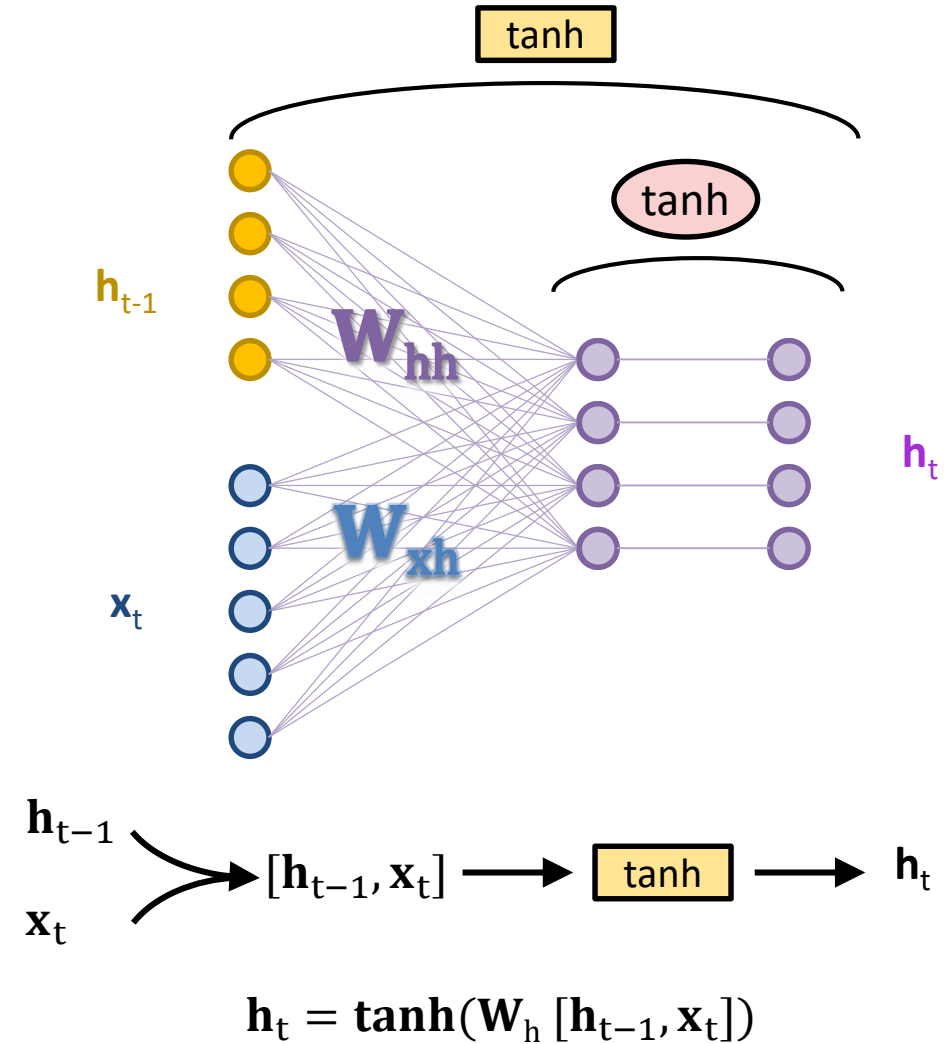
RNN standard

Pour un RNN standard, l'état caché est actualisé par la formule :

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$

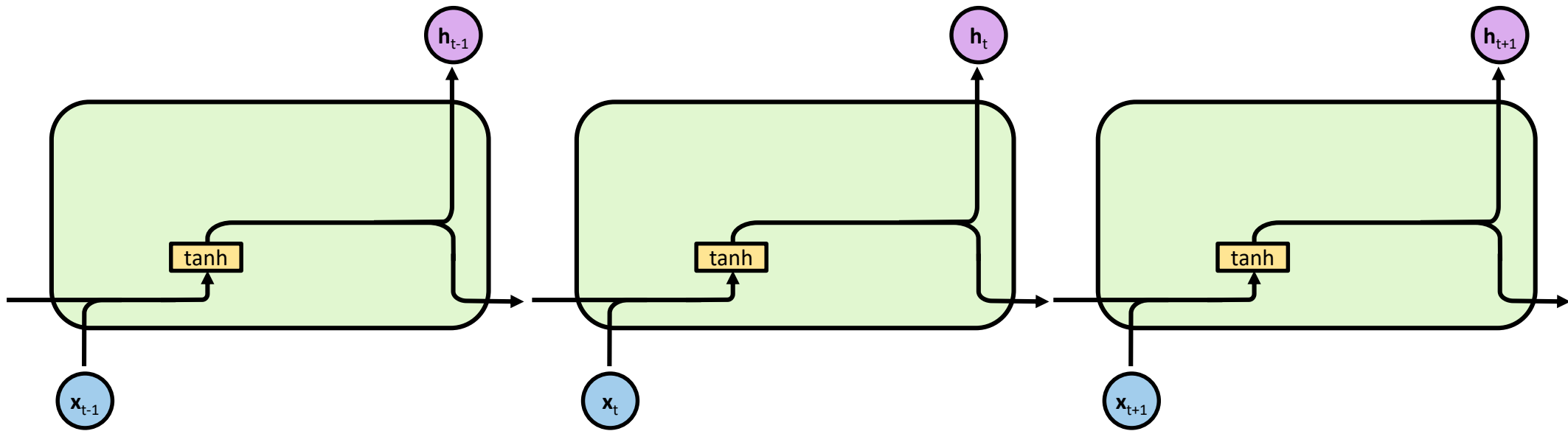


tanh est utilisée pour aider à réguler les valeurs circulant à travers le réseau. La fonction tanh écrase les valeurs pour qu'elles soient toujours comprises entre -1 et 1.



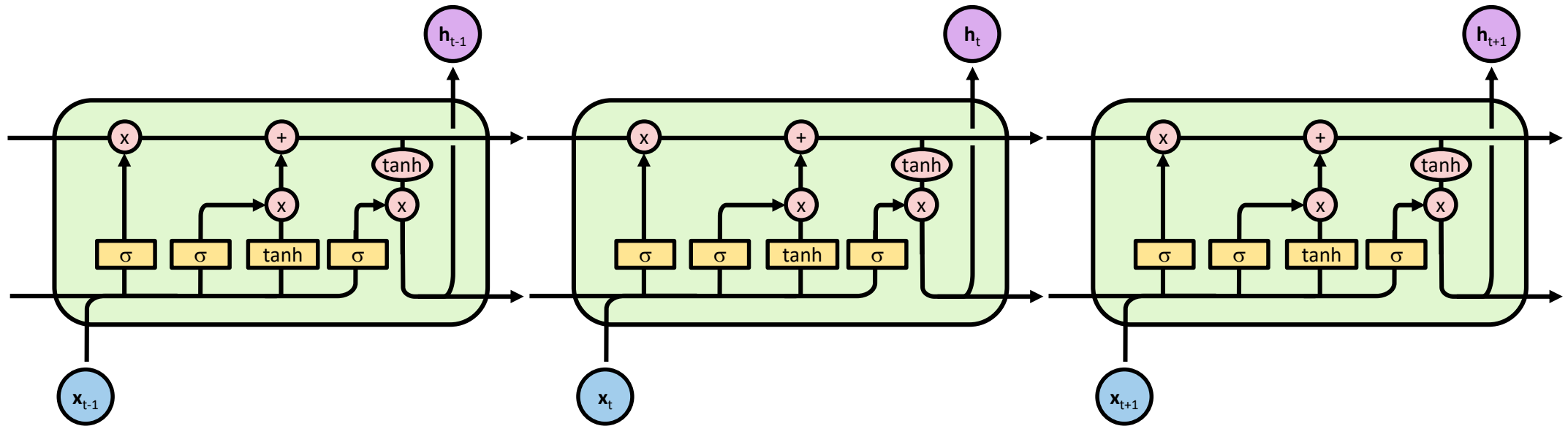
Réseaux RNN

Tous les réseaux de neurones récurrents ont la forme d'une chaîne de modules (ou cellules) répétitifs de réseau de neurones. Dans les RNN standard, ce module répétitif aura une structure très simple, telle qu'une seule couche de $\tanh$.



Réseaux LSTM

Les LSTM ont également cette structure en chaîne, sauf que le module répétitif au lieu d'avoir une seule couche de réseau de neurones dispose de quatre, qui ont des rôles spéciaux que nous allons détailler par la suite.



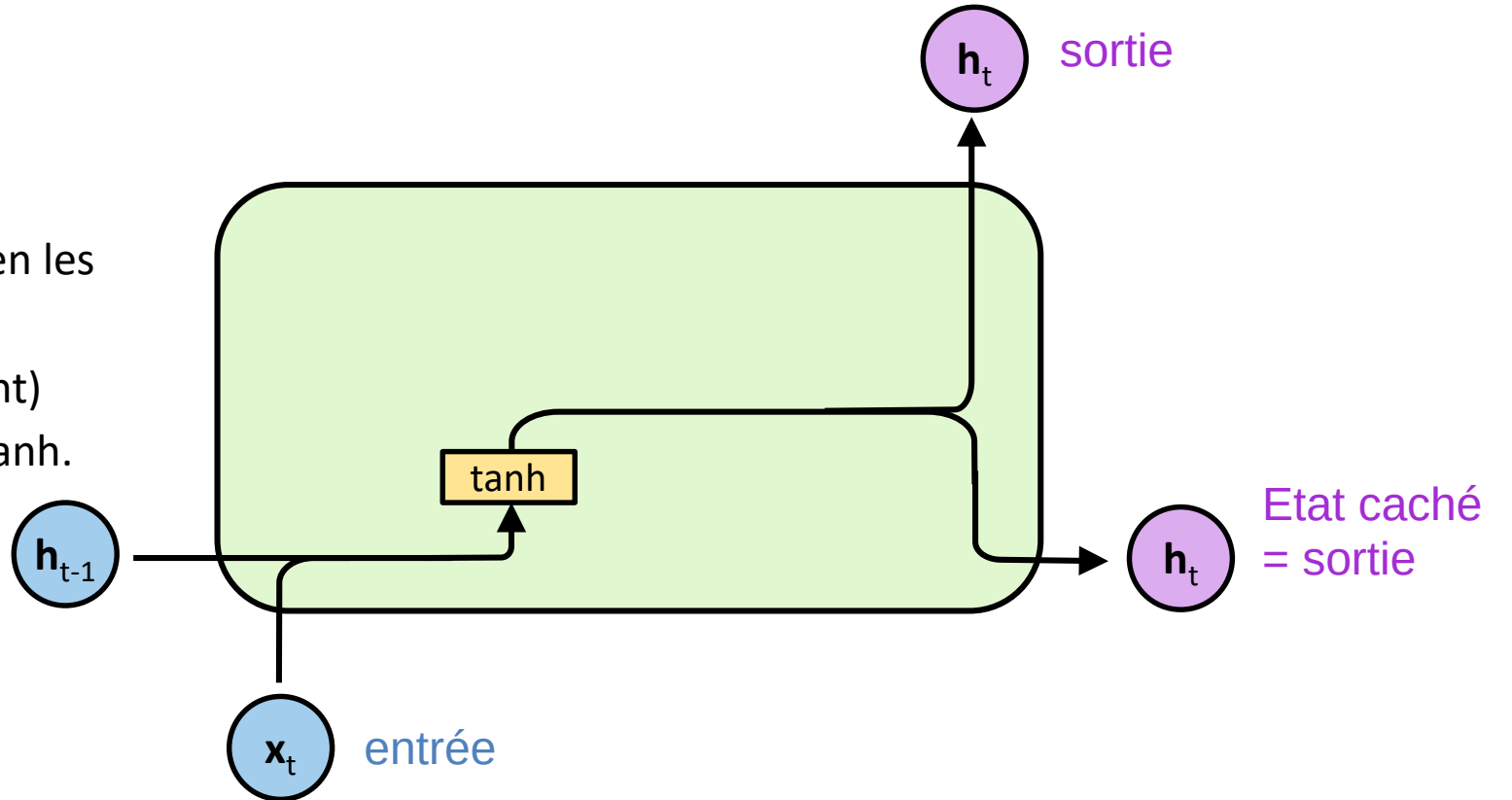
Cellule RNN

Nous allons présenter les LSTM dans une suite de transformations du réseau RNN standard. L'état caché est actualisé par la formule :

Equations

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$

Le plus souvent ce réseau ne gère pas bien les dépendances à long terme à cause de la disparition du gradient (vanishing gradient) comme conséquence de l'utilisation de tanh.



Construire la cellule LSTM

Les LSTM sont explicitement conçus pour éviter le problème de dépendance à long terme. On introduit une nouvelle variable $\mathbf{c}_t$ qui permettra de se souvenir des informations pendant de longues périodes.

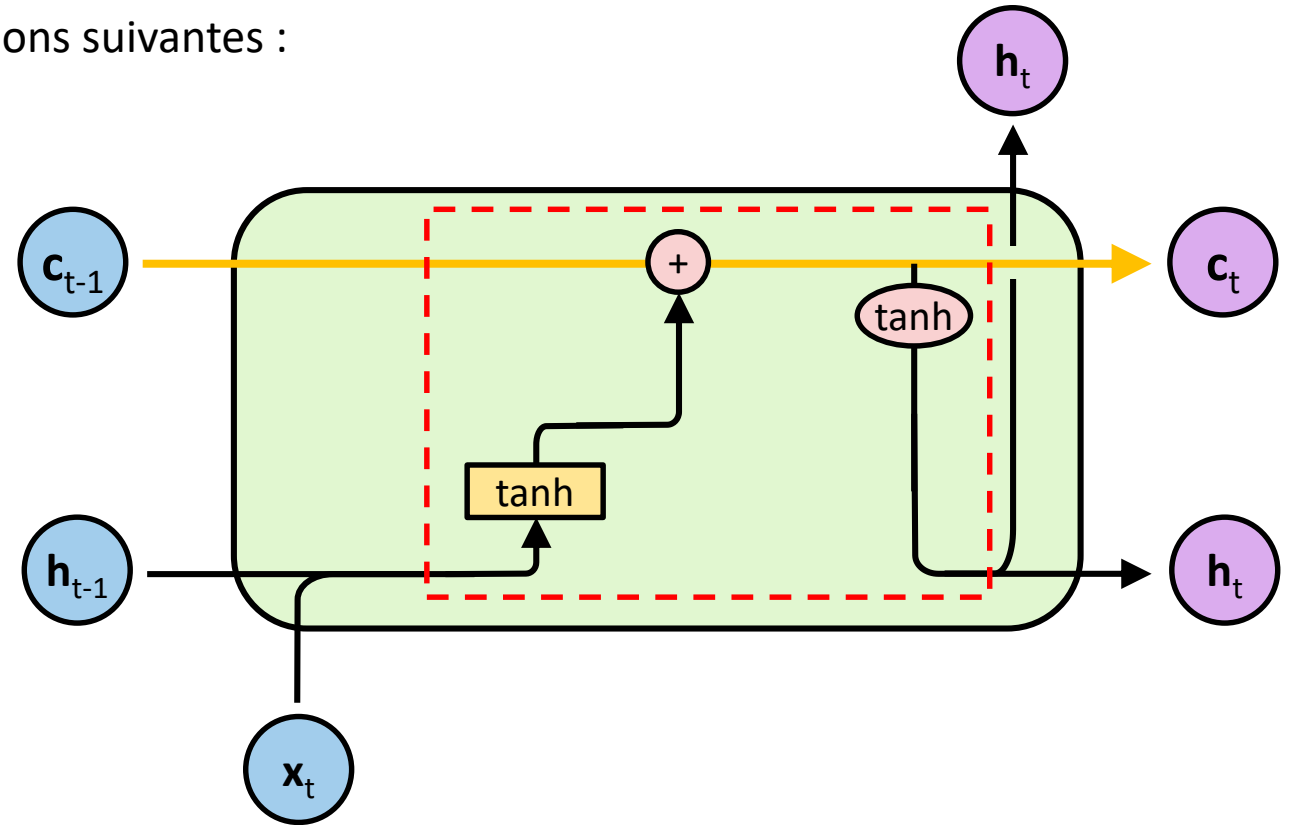
La mise à jour des variables se fera alors selon les équations suivantes :

Equations :

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$

$$\mathbf{h}_t = \tanh(\mathbf{c}_t)$$

Cette fois, on est assuré que l'état caché $\mathbf{h}_t$ calculé à l'instant t tient compte de tous les états cachés précédents.



Construire la cellule LSTM

Les LSTM sont explicitement conçus pour éviter le problème de dépendance à long terme. On introduit une nouvelle variable c_t qui permettra de se souvenir des informations pendant de longues périodes.

c_t est appelée état de la cellule.

La mise à jour des variables se fera alors selon les équations suivantes :

Equations :

$$c_t = c_{t-1} + \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

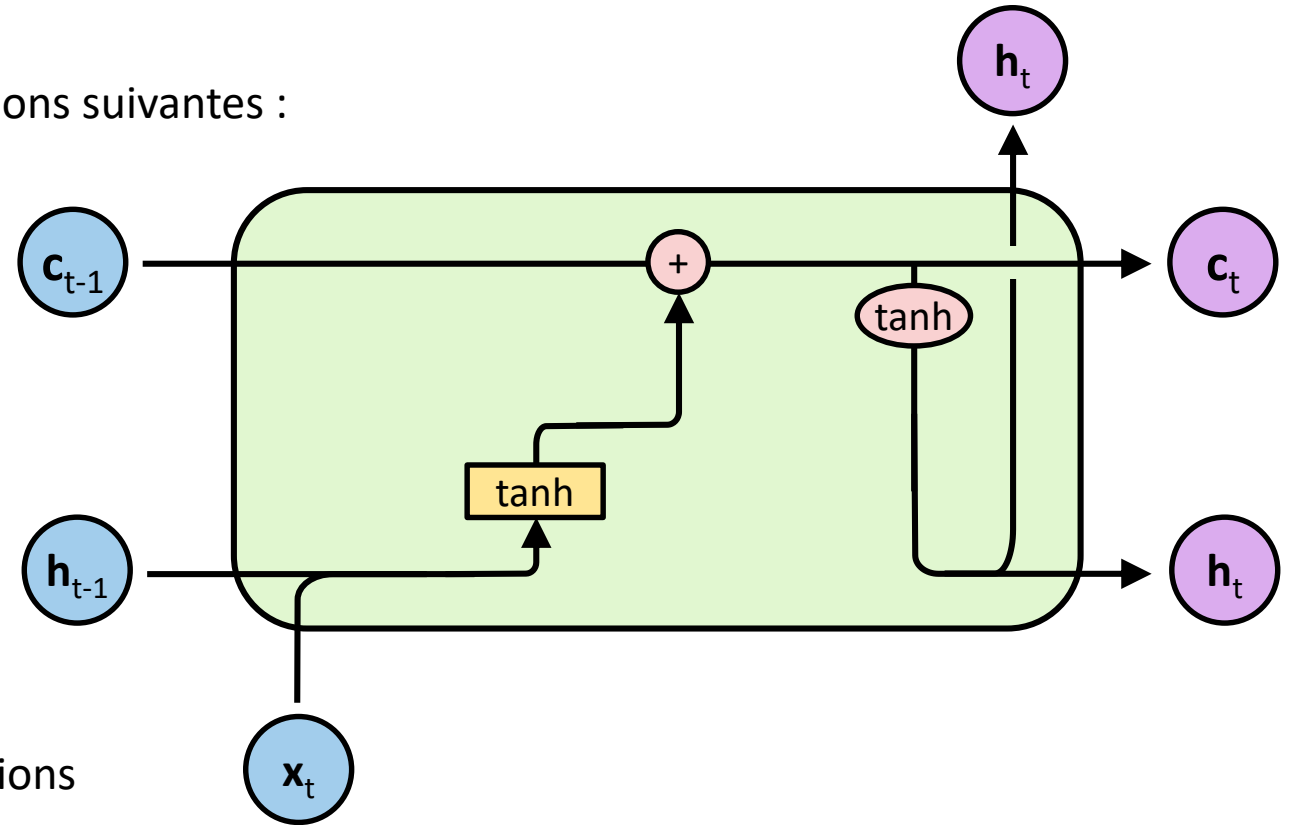
$$h_t = \tanh(c_t)$$

Inconvénient :

Le passé est toujours aussi important que le présent.
Ce qui n'est pas pertinent.

Solution idéale :

Permettre au réseau de garder ou d'oublier les informations du passé selon qu'elles soient déterminantes ou pas.



Construire la cellule LSTM

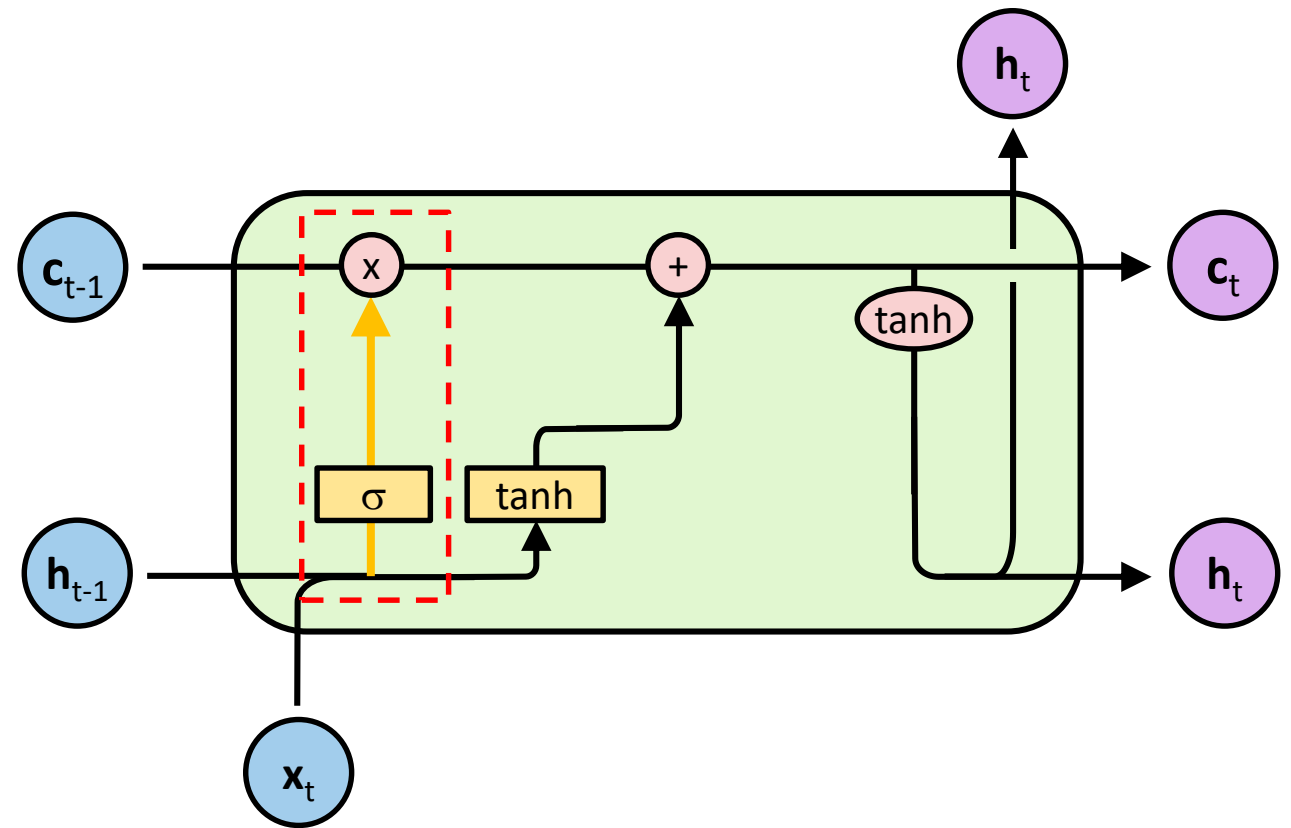
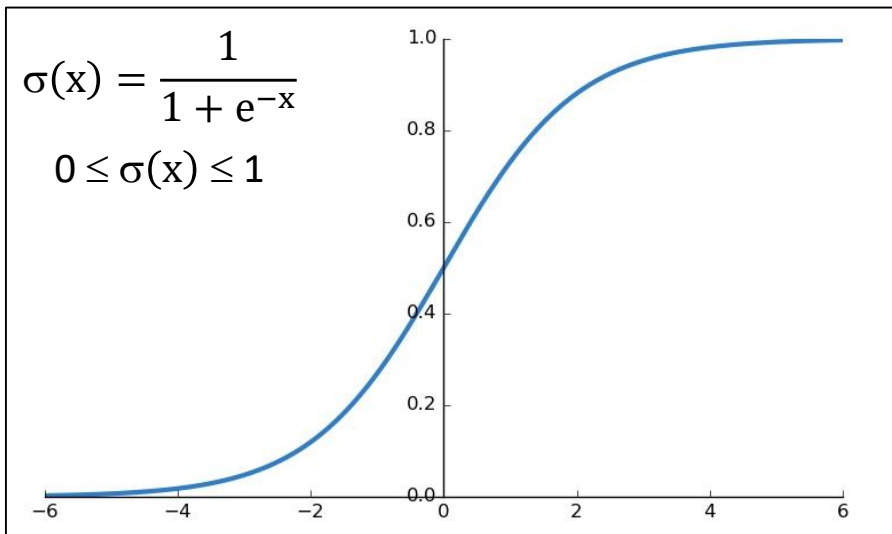
La solution adoptée est de pondérer la mémoire du passé représentée par c_t par un réseau de neurones avec une seule couche sigmoïde. Ce réseau va apprendre à oublier les informations non pertinentes. Il est appelé porte d'oubli ou forget gate.

Equations :

$$f_t = \sigma(W_{hf} h_{t-1} + W_{xf} x_t)$$

$$c_t = f_t * c_{t-1} + \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$h_t = \tanh(c_t)$$



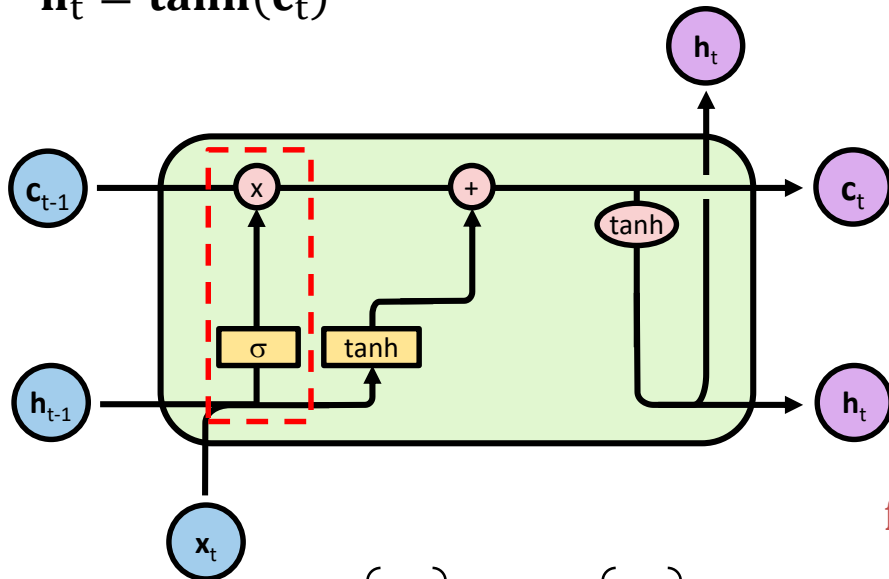
Construire la cellule LSTM

Equations :

$$\mathbf{f}_t = \sigma(\mathbf{W}_{hf} \mathbf{h}_{t-1} + \mathbf{W}_{xf} \mathbf{x}_t)$$

$$\mathbf{c}_t = \mathbf{f}_t * \mathbf{c}_{t-1} + \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$

$$\mathbf{h}_t = \tanh(\mathbf{c}_t)$$



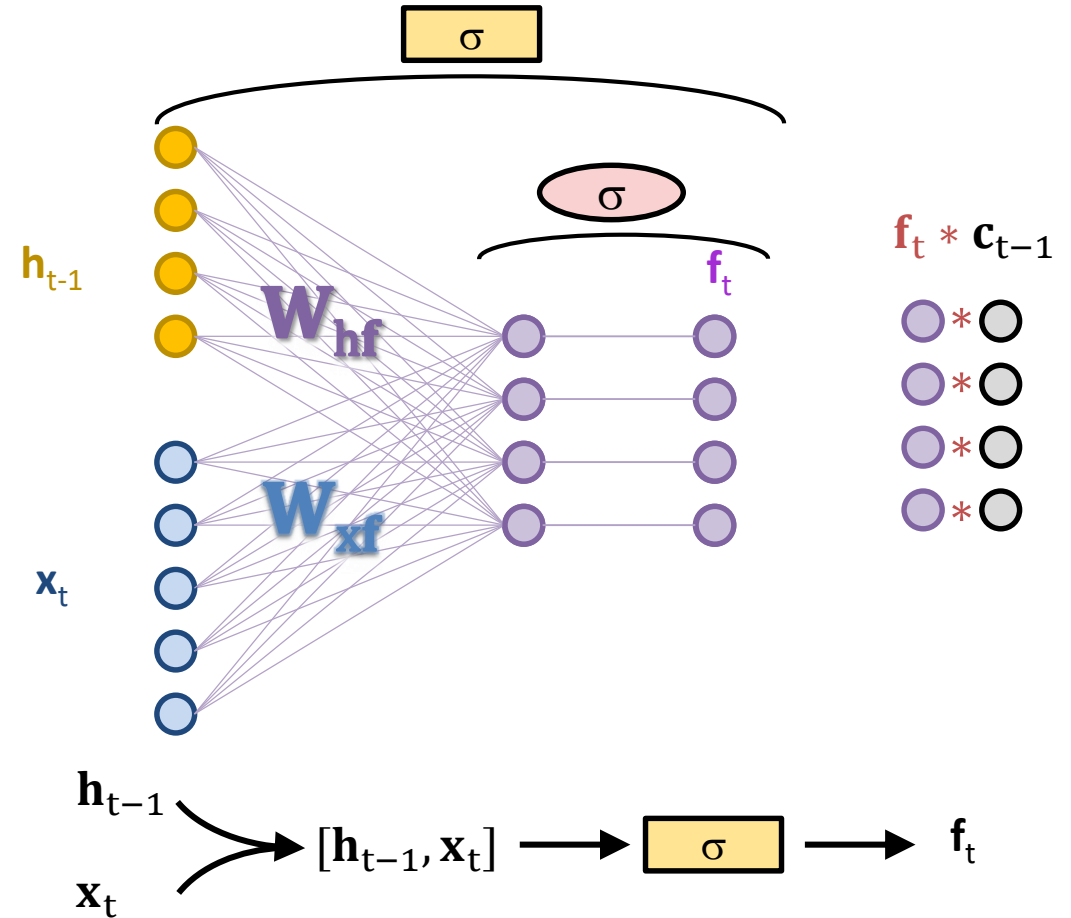
$$\mathbf{c}_{t-1} = \begin{bmatrix} 0.2 \\ -0.4 \\ 0.75 \\ 0.83 \end{bmatrix}$$

$$\mathbf{f}_t = \begin{bmatrix} 0.9 \\ 0.01 \\ 0.3 \\ 0.95 \end{bmatrix}$$



$$\mathbf{c}_t = \begin{bmatrix} 0.18 \\ 0.00 \\ 0.23 \\ 0.79 \end{bmatrix}$$

← oubli



Construire la cellule LSTM

Une autre porte est ajoutée pour pondérer la mise à jour (additive) de l'état de la cellule c_t par la sortie tanh prenant comme entrée h_{t-1} et x_t . Cette porte qui s'appelle input gate va apprendre à utiliser, à ignorer ou à moduler les informations entrées selon leur importance.

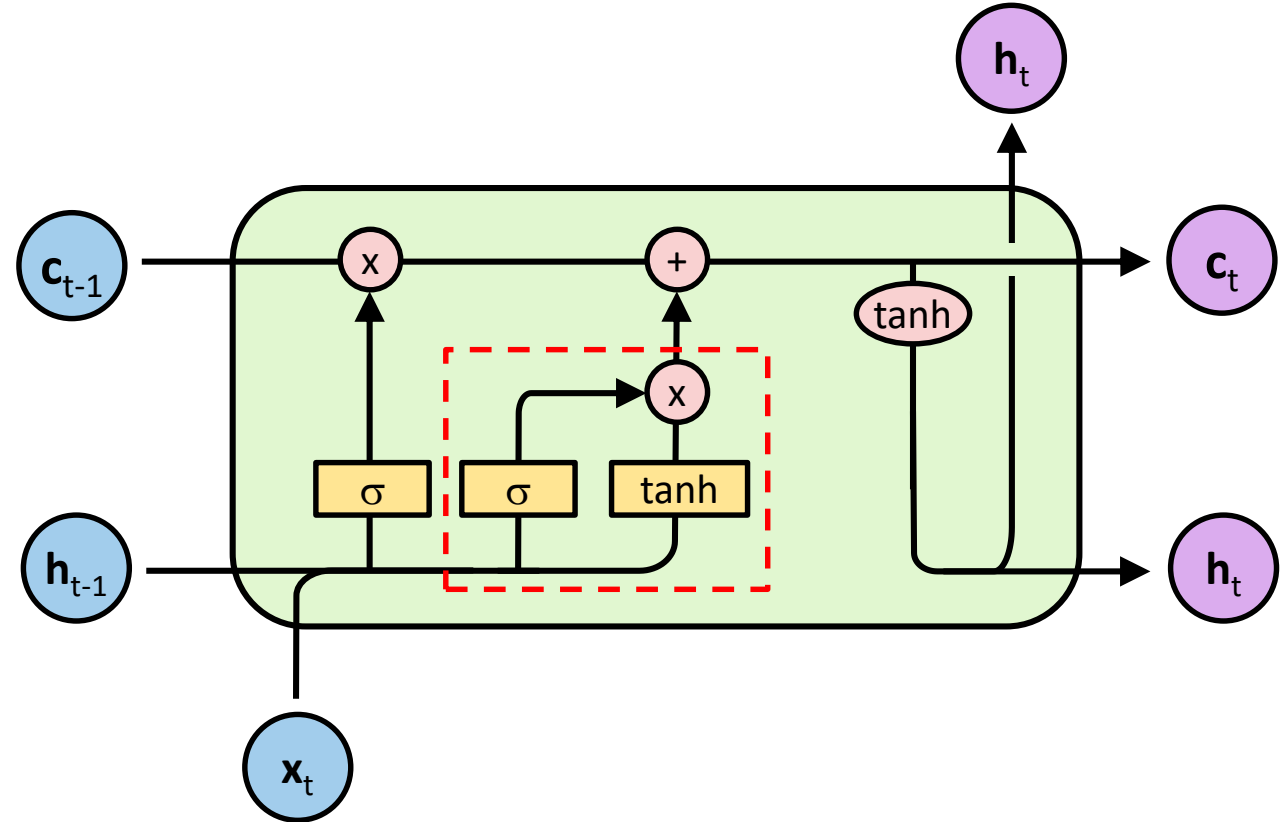
Equations :

$$f_t = \sigma(W_{hf} h_{t-1} + W_{xf} x_t)$$

$$i_t = \sigma(W_{hi} h_{t-1} + W_{xi} x_t)$$

$$c_t = f_t * c_{t-1} + i_t * \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$h_t = \tanh(c_t)$$



Construire la cellule LSTM

De la même manière, une porte de sortie est ajoutée pour pondérer la mise à jour de l'état caché $\mathbf{h}_t$. Cela permet de décider quelles informations l'état caché $\mathbf{h}_t$ doit porter. Cette porte qui s'appelle output gate.

Equations :

$$\mathbf{f}_t = \sigma(\mathbf{W}_{hf} \mathbf{h}_{t-1} + \mathbf{W}_{xf} \mathbf{x}_t)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_{hi} \mathbf{h}_{t-1} + \mathbf{W}_{xi} \mathbf{x}_t)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_{ho} \mathbf{h}_{t-1} + \mathbf{W}_{xo} \mathbf{x}_t)$$

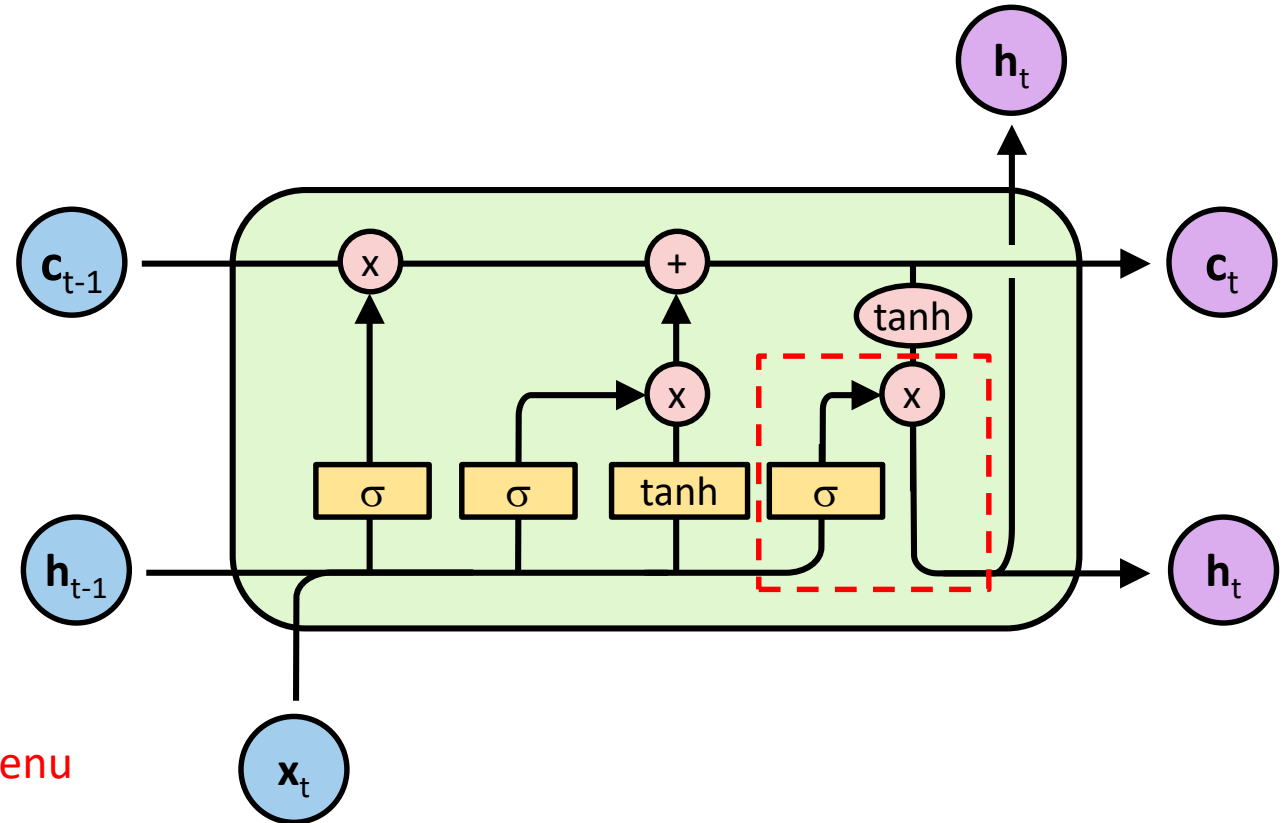
$$\mathbf{c}_t = \mathbf{f}_t * \mathbf{c}_{t-1} + \mathbf{i}_t * \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$

$$\mathbf{h}_t = \mathbf{o}_t * \tanh(\mathbf{c}_t)$$

Etat caché $\mathbf{h}_t$

$$\tanh(\mathbf{c}_t) = \begin{bmatrix} -0.8 \\ -0.3 \\ 0.64 \\ -0.01 \end{bmatrix} \quad \mathbf{o}_t = \begin{bmatrix} 0.96 \\ 0.1 \\ 0.01 \\ 0.38 \end{bmatrix} \Rightarrow \mathbf{h}_t = \begin{bmatrix} -0.77 \\ -0.03 \\ 0.01 \\ 0.00 \end{bmatrix}$$

← retenu
← non retenu



LSTM, une synthèse

Forget gate : $f_t = \sigma(W_{hf} h_{t-1} + W_{xf} x_t)$

Dans le cas extrême, indique à l'état de la cellule (mémoire à long-terme) les informations à oublier (multiplication par 0) ou à conserver (multiplication par 1).

Input gate : $i_t = \sigma(W_{hi} h_{t-1} + W_{xi} x_t)$

déterminer quelles informations produite par la couche tanh doivent entrer dans l'état de la cellule (donc à sauvegarder dans la mémoire à long terme).

Output gate : $o_t = \sigma(W_{ho} h_{t-1} + W_{xo} x_t)$

Indique les informations qui doivent passer au prochain état caché h_t .

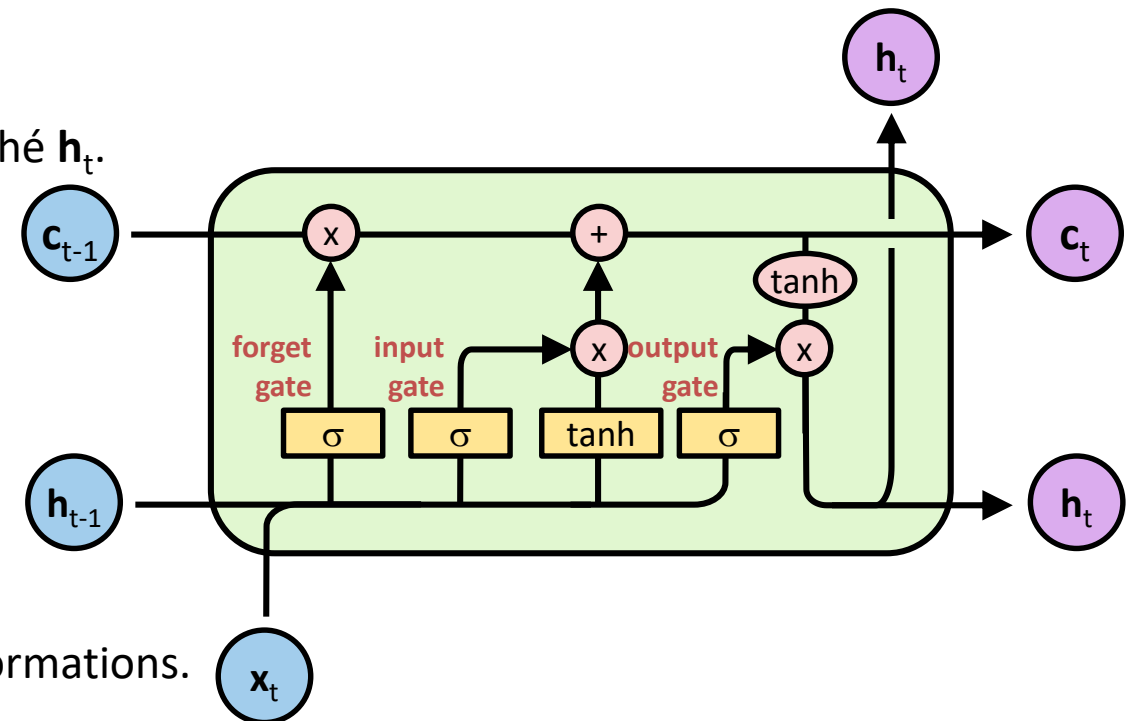
Calcul de c_t et de h_t :

$$c_t = f_t * c_{t-1} + i_t * \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$h_t = o_t * \tanh(c_t)$$

L'état de la cellule c_t parcourt toute la chaîne du réseau avec des interactions linéaires mineures, ce qui leur permet de transporter efficacement les informations.

Les portes (gates) permettent de réguler intelligemment ces informations.



Réseau LSTM

